

24-Python Exception Handling, Binary File Operations, and Introduction to Regex

Introduction

This document is a detailed lecture on advanced Python programming topics, focusing on practical aspects of file handling (with an emphasis on binary files), in-depth exception handling, user-defined exceptions, and an overview of regular expressions (regex).

Binary File Operations in Python

- Binary files (e.g., images, audio, video, PDF) are handled with modes like `rb` (read binary) and `wb` (write binary).
 - To read/write/copy binary files, append 'b' to mode (`rb`, `wb`, etc.). Example provided for copying an image file from `python.jpg` to `new.png` using binary modes.
 - The `pickle` module is introduced for serializing Python objects to binary files with `pickle.dump()` (write) and `pickle.load()` (read).
 - When reading/writing with pickle, objects like lists are converted to binary before storage.
-

Exception Handling in Python

- **Need for Exception Handling:**

- Prevents application crashes by gracefully handling runtime errors (e.g., ATM card not released due to error).
- Compile-time errors (e.g., syntax, indentation) must be fixed by correcting code, while runtime errors can be trapped and managed.

- **Exception Handling Constructs:**

- `try`: Place code that may cause an error.
- `except`: Handles errors from the try block.
- `else`: Code runs if no error occurred in try block.
- `finally`: Always executes, used for cleanup (file/database closure), regardless of errors.
- `raise`: Explicitly raise a custom exception.

- **Compound Exception Handling:**

- Multiple except blocks can handle different error types.
 - Order matters: Always place more specific (child) exceptions before broader (parent) exceptions.
 - Nested try-except blocks are possible, each try must have at least one except.
 - Example provided for handling `ZeroDivisionError` and `ValueError` separately.
-

Types of Exceptions

- Built-in exception hierarchy: `Exception` (parent of all), with subtypes like `ArithmeticError`, `ZeroDivisionError`, `ValueError`, `TypeError`, `IndexError`, `EOFError`, etc.
 - Each exception provides a default error message and can be caught individually or generally.
-

User-Defined Exceptions

• Creation:

- Inherit from Python's built-in `Exception` class.
- Define an `__init__` constructor, always starting with `self` and typically including a `message` parameter.
- Example provided for an `ATM` withdrawal operation, raising an exception when withdrawal amount is too high.
- Custom message is passed and accessible through the exception object.

• Usage:

- Exception is raised with `raise`. Handling is generally done where the exception might be triggered (typically, where the function is called).
- Only one constructor (`__init__`) per class; if multiple custom exceptions needed, define one class per exception.

- **Discussion on `self`:**

- Acts like `this`

in other languages; refers to the instance being constructed and stores attributes for that object.

Exception Handling Rules and Best Practices

- Each try must have at least one except.
 - Multiple excepts allow for specificity; always write child before parent exceptions.
 - try, except, else, finally must follow order; else and finally are optional.
 - Clean-up code should go in finally to avoid resource leaks (e.g., database connections).
 - General (catch-all) excepts can handle unexpected errors, but are less specific and less ideal for custom error handling.
-

Regular Expressions (Regex): Introduction

- **Definition:**

- Pattern-matching tool used for text extraction, searching, and validation.
 - Useful for tasks like validating email/phone number formats, extracting data from text files, and forming complex search patterns.

- **Usage Examples:**

- Find data matching a particular pattern (e.g., email addresses, credit card numbers) in files or entries.

- **Syntax Overview:**

- Regex patterns defined with notations like `\b`, `\w`, `\A`, `\Z`, `{m,n}`, etc.

- Regex operations are often wrapped in raw string literals (e.g., `r'...'`).

- Example pattern to match an email of the form `[A-Za-z0-9]+@[ \w]+\.(com|net|org)` is provided. This matches email IDs containing alphanumeric characters, followed by an '@', a domain name, and ending in '.com', '.net', or '.org'.

- **Assignments:**

- Students are asked to read about regex patterns, methods, and prepare for a deeper exploration in the next class.

Assignments and Interactions

- Students interact with the instructor, asking clarifying questions about where to place try/except blocks, the use of self and constructors, and best practices for custom exception class design.
 - Instructor gives practical assignment: implement further validation (e.g., PAN number for high-value withdrawals with user-defined exceptions), read up on regex, and understand cleanup and error handling in more realistic contexts.
-

Conclusion

- The lecture covers step-by-step operation of binary file IO, exhaustive detail on handling Python exceptions (including best-practice structuring and customization), and initiates a guided introduction to regex for text processing.
- Students are tasked with practical programming exercises and preparation for the next session's regex focus.

Notes powered by [CraftNote](#)